

How does an antivirus engine work?

The first thing that you do when a friend brings in his USB drive chock-full of songs that he just downloaded from (ahem!) a discreet source on the Internet is run a scan for viruses on the drive. Although you just had to click a button, some of the world's most intelligent and complicated code is running in the background to keep your computer safe from harm

We'll be talking about the core of antivirus program i.e. its engine. Most of the modern AV engines are just rehashed versions of existing AV engines, except a few that were written from ground-up. The basic building blocks of an antivirus program are:

- Engine core
- File system interface
- Memory scanner
- Emulators
- Update mechanisms

The engine core is the framework that keeps the entire software together. In essence it acts as the 'glue' and calls the external module as and when required. Usually there are scan engine frameworks already in place such as the Exchange virus protection, that use different scan engines from different vendors, which will be directly called out of the framework. But some antivirus programs utilise their own engines for added benefits along with readily available scan engines like F-Prot or AVP. The engine core contains both signature and heuristics based scanners.

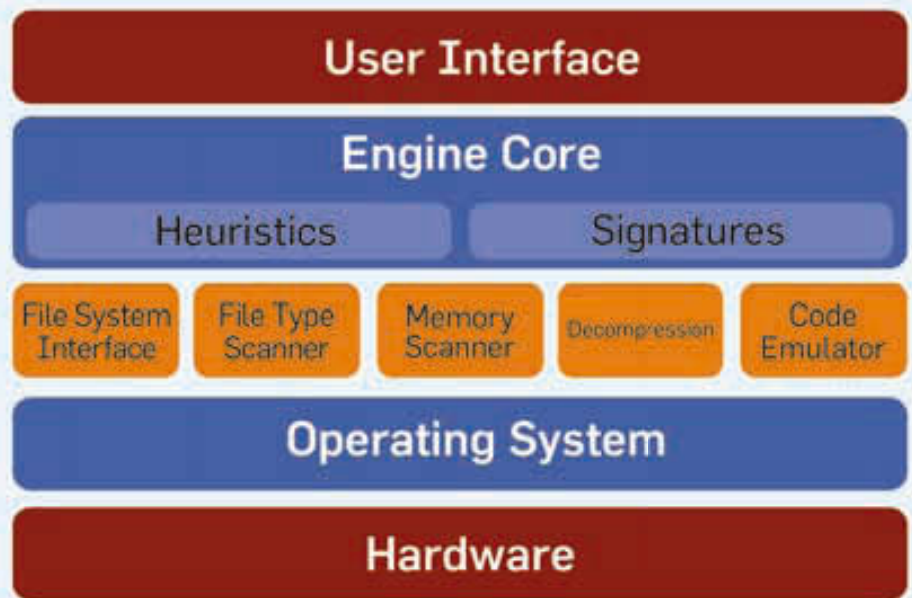
Traditionally, antivirus engines depended on scan string-based technologies. The signature-based scan engine searches within given files for the presence of certain strings often in found in certain regions only. If these predefined strings are found, certain actions like alarms can be triggered. However, signature-based scanning only detects known malware and may not work against new attack mechanisms.

Heuristic scanning is similar to signature-based scanning where instead of looking for specific strings in files it looks for certain instructions or commands within a program that are not found in typical programs. Say for

example in most cases virus are encrypted, so the heuristics scanner looks for a decryption loop and in case it does find a decryption loop it takes the decrypted code and runs it through an emulator to verify whether its actually malicious code or not. As a result of this mechanism heuristics based scanners have more chance

on processing files. This block or module is a major bottleneck and hence is very carefully designed. In most cases there is an abstraction layer between the core AV engine and the file system, which layer has to include conditional compilation for dedicated platforms. The memory scanning block scans the RAM for malicious code, and halts execution of the program in question. But more or less the memory scanning component doesn't fall under the purview of the core engine.

Emulators are used by the engine to check whether the code is malicious or not. It's a virtualized environment in which the code is executed by the AV engine. Based on the instructions running inside the emulator it determines whether it has any malicious functionality or not. Another important



Block diagram of a typical antivirus engine with different functional layers

of detecting previously unexamined, malicious functionality such as the replication mechanism of a virus, the distribution routine of a worm or the payload of a trojan. When you submit a file to the AV for scanning, it goes through a complex series of steps of fetching the file, scanning based on signatures and/or heuristics and then further processing it based on the outcome.

Antivirus software are all about files, in fact the better part of their operations are spent

module is the update module which updates the software so that it can deal with the latest threats, the update mechanism is usually in two part choices of update functionality: update data files or update data files and update executable code. The data files are usually virus definitions or signatures while it's more complicated to update the executable code when the actual modules are replaced. **d**